Self-Improving Multi-Agent Architecture for Tabular Machine Learning Workflows

Technical Architecture Note

1 Purpose

This document defines a self-improving multi-agent architecture for executing end-to-end tabular machine learning workflows. The system is designed to profile datasets, engineer features, benchmark XGBoost and LightGBM models, track experiments with MLflow, and register champion model artifacts while operating under strict sandbox and data-governance constraints.

The architecture adapts the self-improving agent pattern in which learning is performed over complete trajectories rather than static input-output examples. In this pattern, the system first acts in an environment, records the full rollout, evaluates the resulting trajectory, and then uses the reward signal to improve future policy behavior.

$$\text{Act} \rightarrow \text{Record Trajectory} \rightarrow \text{Evaluate} \rightarrow \text{Optimize Policy} \rightarrow \text{Redeploy}$$

For this use case, the environment is not a game or a simple chat interface. The environment is a controlled data science workflow composed of specialized agents, sandboxed execution tools, profiling utilities, experiment tracking systems, and model artifact registries.

2 Pattern Adaptation

The reference pattern can be summarized as:

$$\text{State} \rightarrow \text{Policy} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Reward} \rightarrow \text{Policy Update}$$

We adapt this pattern to a tabular ML workflow as follows:

$$s_t = \{\text{workflow state}, \text{schema metadata}, \text{profiling summary}, \text{agent outputs}, \text{tool results}, \text{MLflow metrics}\}$$

$$a_t \in \{\text{profile dataset}, \text{preprocess features}, \text{select model}, \text{run training}, \text{log metrics}, \text{register model}, \text{review workflow}\}$$

$$r_t = \text{workflow-level reward based on correctness, compliance, reproducibility, and model quality}$$

Thus, the system does not only learn to produce a final answer. It learns a better process for performing data science work.

3 Architecture Overview

The proposed workflow is organized as a layered multi-agent system:

The system contains the following major components:

1. **Supervisor Agent**: Orchestrates the workflow and routes tasks to specialist agents.
2. **Data Engineer Agent**: Extracts data, runs profiling, creates preprocessing logic, and exports clean artifacts.
3. **Gradient Boosting Specialist Agent**: Benchmarks XGBoost and LightGBM models using the cleaned dataset.
4. **Experiment Tracking Agent**: Logs parameters, metrics, and artifacts to MLflow.
5. **Sandbox Execution Agent**: Ensures all code runs inside the Dockerized Code Interpreter.
6. **Reviewer / Critic Agent**: Validates correctness, compliance, and reproducibility.
7. **Agent Lightning**: Captures the complete execution trajectory and attaches reward metadata.
8. **veRL Trainer**: Optimizes the policy using grouped rollouts and GRPO.
9. **QLoRA / LoRA Checkpoint**: Stores the improved policy adapter for redeployment.

4 System Purpose

The system is an advanced, self-improving multi-agent AI platform engineered to execute end-to-end data science workflows on tabular datasets. Its objective is to engineer features, benchmark XGBoost and LightGBM models, track experiments, and register champion models autonomously while operating strictly within a sandboxed environment.

The workflow is designed around four operational tools:

1. **SQLAlchemy Connector**: Used only to securely extract raw tabular data from databases or write back processed tables.
2. **YData Profiling**: Used to generate schema, distribution, missing-value, cardinality, and target-balance summaries.
3. **MLflow Tracking**: Used to initialize experiments, log hyperparameters, log metrics, and register final model artifacts.
4. **Dockerized Code Interpreter**: Used as the only approved runtime for generated Pandas, Polars, XGBoost, LightGBM, and MLflow scripts.

5 Strict Operational Guardrails

The system is governed by three primary constraints:

5.1 No Raw Data in Context

Raw records, full dataframes, and complete tables must never be inserted into the LLM context.

$$C_{raw} = 0$$

where C_{raw} represents the number of raw data rows exposed to the language model.
The LLM may inspect only:

- schema definitions,
- YData profiling summaries,
- missing-value reports,
- cardinality summaries,
- target distribution summaries,
- execution logs,
- MLflow metrics,
- artifact paths.

5.2 Sandbox Compliance

Every generated script, shell command, preprocessing routine, model-training job, or MLflow operation must execute inside the Dockerized Code Interpreter.

$$E_{host} = 0$$

where E_{host} represents direct execution outside the sandbox.

5.3 Schema Metadata Contract

The Gradient Boosting Specialist Agent must parse the exact schema metadata exported by the Data Engineer Agent.

$$S_{train} = S_{exported}$$

This prevents structural mismatches between preprocessing and training.

6 Agent Roles

6.1 Supervisor Agent

The Supervisor Agent is responsible for orchestration.
Its responsibilities are:

- understand the user request,
- request missing inputs,
- identify the target column and task type,
- route work to the Data Engineer Agent,
- route cleaned artifacts to the Gradient Boosting Specialist Agent,
- ensure MLflow tracking is enabled,
- invoke the Reviewer / Critic Agent before final response,
- ensure all guardrails remain active.

The Supervisor Agent begins by requesting:

- SQLAlchemy connection string or dataset file path,
- table name or SQL query,
- target column,
- task type,
- preferred primary metric,
- MLflow tracking URI if already configured.

6.2 Data Engineer Agent

The Data Engineer Agent performs Phase 1 of the workflow.

$$A_{DE} : D_{raw} \rightarrow P_{profile} \rightarrow D_{clean} \rightarrow S_{metadata}$$

where:

$$D_{raw} = \text{raw dataset}$$

$$P_{profile} = \text{YData profiling summary}$$

$$D_{clean} = \text{clean Parquet artifact}$$

$$S_{metadata} = \text{schema metadata JSON}$$

The Data Engineer Agent must:

1. connect to the dataset through SQLAlchemy or file path;
2. generate a YData Profiling report;
3. read the text-based profiling summary;
4. detect missing values;
5. detect high-cardinality categorical columns;
6. detect numerical, categorical, and datetime columns;
7. detect target imbalance;
8. identify potential leakage columns;
9. generate preprocessing code;
10. execute preprocessing only in the Dockerized Code Interpreter;
11. save a clean Parquet file;
12. export schema metadata as JSON.

6.3 Gradient Boosting Specialist Agent

The Gradient Boosting Specialist Agent performs Phase 2 of the workflow.

It consumes:

$$D_{clean}, S_{metadata}, P_{profile}$$

and produces:

$$M_{champion}, R_{mlflow}$$

where:

$$M_{champion} = \text{best registered model artifact}$$

$$R_{mlflow} = \text{MLflow experiment run record}$$

The model selection rule is:

$$M = \begin{cases} \text{LightGBM}, & n > 1{,}000{,}000 \\ \text{LightGBM}, & \text{high-cardinality categorical features exist} \\ \text{XGBoost}, & \text{otherwise} \end{cases}$$

where n is the dataset row count.
The Gradient Boosting Specialist Agent must:

1. load the clean Parquet file inside the sandbox;
2. parse the exported schema metadata JSON;
3. validate feature columns against the schema contract;
4. prepare cross-validation splits;
5. choose LightGBM or XGBoost using the stated rule;
6. run tuning loops;
7. log parameters and metrics to MLflow;
8. save the optimized model artifact;
9. register the champion model.

6.4 Experiment Tracking Agent

The Experiment Tracking Agent ensures reproducibility.
It logs:

- experiment name,
- run ID,
- dataset version or hash,
- profiling report path,
- schema metadata path,
- preprocessing configuration,
- model type,
- hyperparameters,

- evaluation metrics,
- model artifact URI,
- registered model name and version.

For classification tasks, expected metrics include:

$$\text{AUC}, \quad \text{log-loss}$$

For regression tasks, expected metrics include:

$$\text{RMSE}, \quad \text{MAE}, \quad R^2$$

6.5 Reviewer / Critic Agent

The Reviewer / Critic Agent evaluates the complete workflow, not just the final model metric. It scores the trajectory along the following dimensions:

$$Q = \{q_{profile}, q_{preprocess}, q_{schema}, q_{model}, q_{tracking}, q_{sandbox}, q_{guardrail}, q_{reproducibility}, q_{metric}\}$$

where:

$$\begin{aligned}
q_{profile} &= \text{profiling quality} \\
q_{preprocess} &= \text{preprocessing quality} \\
q_{schema} &= \text{schema integrity} \\
q_{model} &= \text{model selection correctness} \\
q_{tracking} &= \text{MLflow tracking completeness} \\
q_{sandbox} &= \text{sandbox compliance} \\
q_{guardrail} &= \text{raw-data guardrail compliance} \\
q_{reproducibility} &= \text{reproducibility} \\
q_{metric} &= \text{final model quality}
\end{aligned}$$

7 Workflow Phases

7.1 Phase 1: Data Engineering

The Data Engineer Agent executes the following sequence:

$$\text{Connect} \rightarrow \text{Profile} \rightarrow \text{Summarize} \rightarrow \text{Preprocess} \rightarrow \text{Export Parquet} \rightarrow \text{Export Schema Metadata}$$

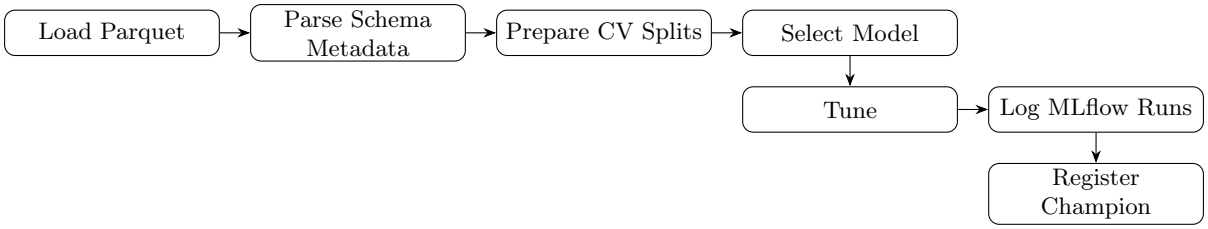
The output contract is:

```
{
  "profile_summary": {
    "row_count": 0,
    "column_count": 0,
```

```
    "target_column": "...",
    "target_distribution_summary": "...",
    "missing_value_summary": "...",
    "high_cardinality_columns": [],
    "numeric_columns": [],
    "categorical_columns": [],
    "datetime_columns": [],
    "potential_leakage_columns": []
  },
  "preprocessing_plan": {
    "missing_value_strategy": {},
    "encoding_strategy": {},
    "scaling_strategy": {},
    "dropped_columns": [],
    "feature_generation": []
  },
  "artifacts": {
    "clean_parquet_path": "...",
    "schema_metadata_path": "...",
    "profile_report_path": "..."
  },
  "execution_status": "success",
  "issues": []
}
```

7.2 Phase 2: Gradient Boosting Benchmarking

The Gradient Boosting Specialist Agent executes:



The output contract is:

```
{
  "model_selection_decision": {
    "selected_model": "LightGBM",
    "reason": "Dataset contains high-cardinality categorical columns.",
    "row_count": 0,
    "high_cardinality_detected": true
  },
  "cv_strategy": {
    "split_type": "stratified_kfold",
    "num_folds": 5,
    "stratified": true
  },
  "mlflow_runs": [
    {
      "run_id": "...",
```

```
      "model_type": "...",
      "params": {},
      "metrics": {
        "auc": 0.0,
        "log_loss": 0.0
      }
    }
  ],
  "champion_model": {
    "model_type": "...",
    "run_id": "...",
    "artifact_path": "...",
    "registered_model_name": "...",
    "primary_metric": "...",
    "primary_metric_value": 0.0
  },
  "execution_status": "success",
  "issues": []
}
```


8 Trajectory Design

Following the self-improving agent pattern, every execution is captured as a trajectory.

$$\tau = \{x, a_1, o_1, a_2, o_2, \ldots, a_T, o_T, r\}$$

where:

$$x = \text{user request}$$

$$a_t = \text{agent action at step } t$$

$$o_t = \text{tool result, artifact, metric, or observation}$$

$$r = \text{final reward}$$

For the tabular ML workflow, a trajectory is:

$$\tau_{ML} = \{x, P, S, D, F, M, L, V, r\}$$

where:

$$
\begin{aligned}
x &= \text{user request} \\
P &= \text{profiling summary} \\
S &= \text{schema metadata} \\
D &= \text{data preprocessing decisions} \\
F &= \text{feature engineering configuration} \\
M &= \text{model training decisions} \\
L &= \text{MLflow logs and artifacts} \\
V &= \text{reviewer validation} \\
r &= \text{trajectory reward}
\end{aligned}
$$

9 Agent Lightning Telemetry

Agent Lightning captures the execution trace and converts it into a trainable trajectory.


```
{
  "task_id": "tabular-ml-001",
  "workflow_type": "tabular_ml",
  "user_request": "Train a binary classifier and track experiments.",
  "trajectory": [
    {
      "step": 1,
      "agent": "SupervisorAgent",
      "state": "received_user_request",
      "action": "requested_connection_string_and_target_column",
      "observation": "user_provided_inputs",
      "memory_reads": [],
      "memory_writes": ["workflow_state"],
      "tool_calls": [],
      "status": "success"
    },
    {
      "step": 2,
      "agent": "DataEngineerAgent",
      "state": "dataset_available",
      "action": "ran_ydata_profiling",
      "observation": "profile_summary_generated",
      "memory_reads": ["workflow_state"],
      "memory_writes": ["schema_metadata", "profile_summary"],
      "tool_calls": [
        "SQLAlchemyConnector",
        "YDataProfiling",
        "DockerizedCodeInterpreter"
      ],
      "status": "success"
    },
    {
      "step": 3,
      "agent": "GradientBoostingSpecialistAgent",
      "state": "clean_parquet_available",
```

```
      "action": "trained_lightgbm_or_xgboost",
      "observation": "mlflow_metrics_logged",
      "memory_reads": ["schema_metadata", "profile_summary"],
      "memory_writes": ["training_summary", "champion_model"],
      "tool_calls": [
        "DockerizedCodeInterpreter",
        "MLflowTracking"
      ],
      "status": "success"
    },
    {
      "step": 4,
      "agent": "ReviewerCriticAgent",
      "state": "workflow_outputs_available",
      "action": "validated_workflow_and_generated_reward_metadata",
      "observation": "approved",
      "memory_reads": [
        "workflow_state",
        "schema_metadata",
        "training_summary"
      ],
      "memory_writes": [
        "reviewer_scores",
        "reward_metadata"
      ],
      "tool_calls": [],
      "status": "success"
    }
  ],
  "metrics": {
    "num_steps": 4,
    "num_tool_calls": 5,
    "num_retries": 0,
    "raw_data_leakage": false,
    "sandbox_violation": false,
    "schema_mismatch": false,
    "auc": 0.91,
    "log_loss": 0.27
  },
  "reward_metadata": {
    "task_success": 1,
    "model_selection_correct": true,
    "mlflow_complete": true,
    "champion_registered": true,
    "needs_retry": false
  }
}
```


10 Reward Function

The trajectory reward is defined as:

$$R(\tau) = \alpha R_{data} + \beta R_{model} + \gamma R_{tracking} + \delta R_{sandbox} + \eta R_{reproducibility} - \lambda R_{violation}$$

where:

$$\begin{aligned} R_{data} &= \text{profiling and preprocessing quality} \\ R_{model} &= \text{model selection and metric quality} \\ R_{tracking} &= \text{MLflow logging completeness} \\ R_{sandbox} &= \text{sandbox compliance} \\ R_{reproducibility} &= \text{artifact and schema traceability} \\ R_{violation} &= \text{raw-data leakage, schema mismatch, or unsafe execution} \end{aligned}$$

A practical reward implementation is:

$$R(\tau) = 0.20R_{data} + 0.20R_{model} + 0.20R_{tracking} + 0.15R_{sandbox} + 0.15R_{reproducibility} - 0.10R_{violation}$$

where $R(\tau) \in [-1, 1]$.

11 Relative Ranking for Grouped Rollouts

For a given ML task x_i, the system generates multiple rollout attempts:

$$G_i = \{\tau_{i,1}, \tau_{i,2}, \ldots, \tau_{i,k}\}$$

Each rollout may make different decisions:

- one may skip profiling,
- one may choose the wrong model family,
- one may fail to log MLflow artifacts,
- one may correctly profile, preprocess, train, track, and register the model.

The system ranks the trajectories relative to each other:

$$R(\tau_{i,3}) > R(\tau_{i,2}) > R(\tau_{i,1})$$

The group-normalized advantage is:

$$A_{i,j} = \frac{R(\tau_{i,j}) - \mu(G_i)}{\sigma(G_i) + \epsilon}$$

where:

$$\mu(G_i) = \text{mean reward of rollout group}$$

$$\sigma(G_i) = \text{standard deviation of rewards in the group}$$

This follows the relative evaluation pattern: instead of asking whether one trajectory is good in isolation, the system asks which trajectory is better among multiple attempts for the same task.

12 GRPO Optimization

The veRL Trainer consumes grouped trajectories and applies Group Relative Policy Optimization.

The objective is:

$$\mathcal{L}_{GRPO}(\theta) = -\mathbb{E}\left[A_{i,j} \log \pi_\theta(\tau_{i,j})\right] + \beta D_{KL}\left(\pi_\theta \| \pi_{ref}\right)$$

where:

$$\pi_\theta = \text{current trainable policy}$$

$$\pi_{ref} = \text{reference policy}$$

$$A_{i,j} = \text{group-normalized advantage}$$

$$D_{KL} = \text{policy drift penalty}$$

The KL term prevents the updated policy from drifting too aggressively away from the reference model.

13 QLoRA Adapter Training

To make training practical on constrained infrastructure, the base model is frozen and only low-rank adapter weights are updated.

$$W' = W + \Delta W$$

$$\Delta W = BA$$

where:

$$W = \text{frozen base weight matrix}$$

$$B, A = \text{trainable low-rank adapter matrices}$$

With QLoRA, the base model is quantized while adapter weights remain trainable:

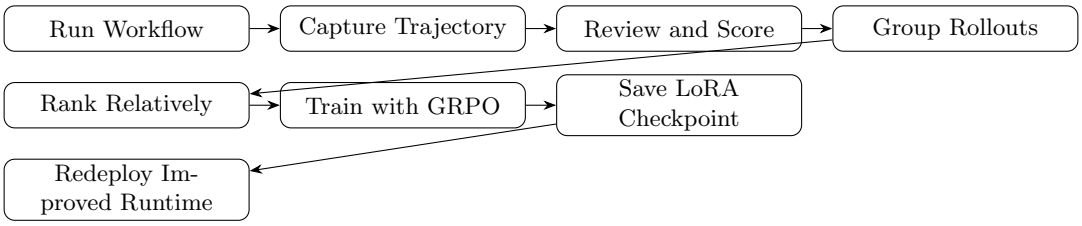
$$W \approx Q_4(W)$$

The training process becomes:

Base LLM → 4-bit Quantization → LoRA Adapter → GRPO Update → Improved Adapter Checkpoint

14 Self-Improving Flywheel

The complete improvement loop is:



This converts a static automated ML workflow into a self-improving data science system.

15 Final Thesis

A traditional AutoML system optimizes models. A self-improving multi-agent ML workflow optimizes the full process that creates those models.

Do not train only on final model metrics. Train on complete workflow trajectories.

The proposed architecture improves not only prediction performance, but also:

- profiling discipline,
- preprocessing correctness,
- feature engineering quality,
- model selection behavior,
- MLflow reproducibility,
- sandbox compliance,
- raw-data governance,
- reviewer-driven quality control.

Thus, the system learns how to perform better data science workflows over time.